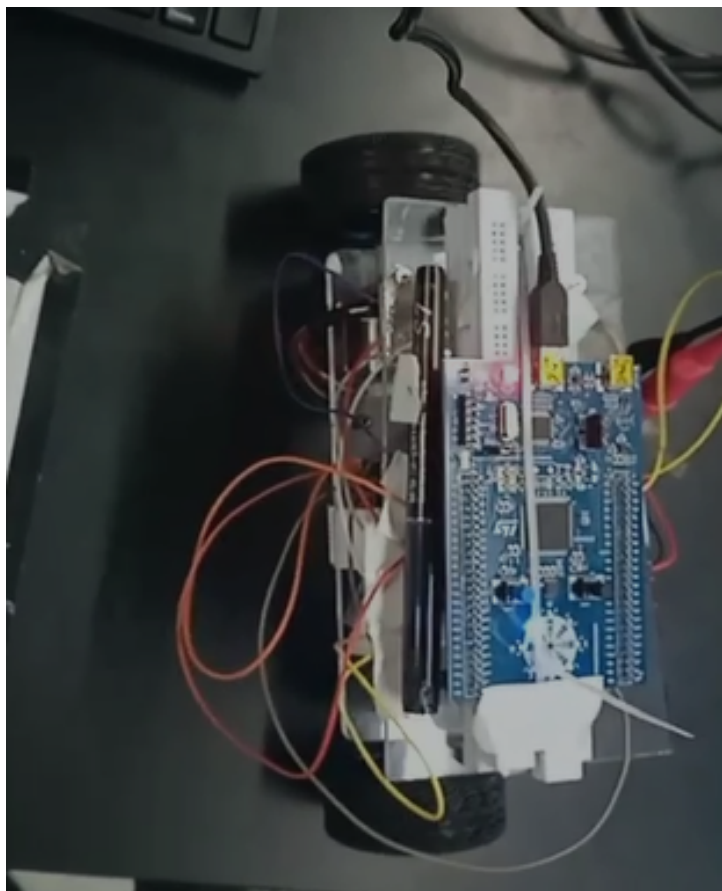


Self-Balancing Robot

Using STM32 Discovery Board
With Bluetooth Data Visualization



Project Report

April 28, 2026

Micro-controllers and Interfacing

Submitted By:

Sualeh Abbas Rizvi
Burair Haidar Inayatullah

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	System Architecture	2
1.3	Special Feature: Bluetooth Visualization	2
2	Theoretical Background	3
2.1	Complementary Filter	3
2.2	PID Control	3
3	Implementation Details	4
3.1	System Parameters	4
3.2	Motor Driver Configuration	4
3.3	Control Logic	5
3.4	Bluetooth Communication Variables	6
3.5	Gyroscope and Accelerometer Reading	6
3.6	Sensor Calibration	7
3.7	Normal Operation: Complementary Filter and PID	8
3.8	UART Data Printing	9
4	Results and Analysis	11
4.1	System Performance	11
4.2	Sensor Signal Analysis	11
4.3	Bluetooth Data Transmission	13
5	Discussion	15
5.1	Design Choices	15
5.1.1	Complementary Filter vs. Kalman Filter	15
5.1.2	PID Control Parameters	15
5.1.3	Bluetooth Update Rate	15
6	System Flowchart and Execution Flow	16
6.1	Overview of Control Flow	16
6.2	Step-by-Step Flow Description	16
6.3	Flowchart Diagram	18
6.4	Key Observations from Flow	19
6.5	Challenges and Solutions	19
6.5.1	ISR Timing Issues	19
6.5.2	Integral Windup	19
6.5.3	Derivative Noise	19
6.6	Limitations	19
7	Evaluation Questions	20
7.1	Effect of PID Gains on System Performance	20
7.2	Challenges in Initial Tuning and Final Strategy	20
7.3	Limitations of Simple PID in Real-World Systems	21

8 Conclusion	21
8.1 Lessons Learned	22
Appendix	23

1 Introduction

1.1 Project Overview

A self-balancing robot is an inverted pendulum system that maintains its upright position through continuous feedback control. This project implements such a system using the STM32F3 Discovery Board, which features:

- STM32F303VCT6 ARM Cortex-M4 microcontroller
- LSM303DLHC 3-axis accelerometer
- I3G4250D 3-axis gyroscope
- Built-in PWM capabilities for motor control

1.2 System Architecture

The system consists of three main components:

1. **Sensor Subsystem:** Gyroscope and accelerometer provide orientation data
2. **Control Subsystem:** PID controller processes sensor data and generates motor commands
3. **Communication Subsystem:** Bluetooth module (HC-06) transmits data to Android app

1.3 Special Feature: Bluetooth Visualization

A key enhancement in this implementation is the integration of wireless data monitoring. The HC-06 Bluetooth module transmits real-time sensor data (filtered tilt angle, gyroscope rate, accelerometer angle) to a custom Android application at 10 Hz. This allows for:

- Real-time visualization of sensor signals
- Monitoring of complementary filter performance
- Analysis of system stability
- Remote debugging and tuning

2 Theoretical Background

2.1 Complementary Filter

The complementary filter combines gyroscope and accelerometer data to produce an accurate tilt angle estimate:

$$\theta_{filtered} = \alpha(\theta_{prev} + \omega_{gyro} \cdot \Delta t) + (1 - \alpha) \cdot \theta_{accel} \quad (1)$$

where:

- $\alpha = 0.98$ (gyroscope weight)
- $(1 - \alpha) = 0.02$ (accelerometer weight)
- ω_{gyro} is the angular velocity from the gyroscope
- θ_{accel} is the angle calculated from accelerometer
- $\Delta t = 0.005$ s (sampling time at 200 Hz)

Rationale:

- The gyroscope provides smooth, high-frequency response but drifts over time
- The accelerometer is immune to drift but is noisy and affected by vibrations
- The complementary filter trusts the gyroscope for short-term changes and uses the accelerometer to correct long-term drift

2.2 PID Control

The PID controller generates motor commands based on the error between setpoint and current angle:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (2)$$

where:

- $e(t) = \theta_{setpoint} - \theta_{filtered}$ is the error
- $K_p = 40.0$ (proportional gain)
- $K_i = 0.1$ (integral gain)
- $K_d = 1.5$ (derivative gain)

Control Terms:

- **Proportional:** Provides immediate correction proportional to current error
- **Integral:** Eliminates steady-state error by accumulating past errors
- **Derivative:** Dampens oscillations by reacting to the rate of error change

3 Implementation Details

3.1 System Parameters

Table 1: Control System Parameters

Parameter	Value	Description
Sampling Time (DT)	0.005 s	200 Hz control loop
K_p	40.0	Proportional gain
K_i	0.1	Integral gain
K_d	1.5	Derivative gain
Setpoint	0.0°	Target tilt angle
PWM Range	±999	Max motor command
Integral Limits	±200	Anti-windup bounds

Table 2: Complementary Filter Weights

Component	Weight
Gyroscope	0.98
Accelerometer	0.02

3.2 Motor Driver Configuration

The motor driver uses H-bridge topology controlled by GPIO pins and PWM signals. Each motor requires two direction pins and one PWM speed control pin.

```

1 void Motor_Left_Forward(uint16_t speed) {
2     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_SET);    // PE7 =
        HIGH
3     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_RESET);  // PE6 =
        LOW
4     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, speed);
5 }
6 void Motor_Left_Backward(uint16_t speed) {
7     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_RESET);  // PE7 =
        LOW
8     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_SET);    // PE6 =
        HIGH
9     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, speed);
10 }
11 void Motor_Left_Stop(void) {
12     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_SET);
13     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_SET);
14     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
15 }
16 void Motor_Right_Forward(uint16_t speed) {
17     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_SET);    // PE8 =
        HIGH
18     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_RESET); // PE12 =
        LOW
19     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, speed);
20 }

```

```

21 void Motor_Right_Backward(uint16_t speed) {
22     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_RESET); // PE8 =
        LOW
23     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_SET); // PE12 =
        HIGH
24     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, speed);
25 }
26 void Motor_Right_Stop(void) {
27     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_SET);
28     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_SET);
29     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
30 }

```

Listing 1: Motor Control Functions

Explanation:

- **Direction Control:** Four GPIO pins (PE7, PE6) and (PE8, PE12) control motor direction through the H-bridge for both motors.
- **Forward:** PE7=HIGH and PE8 = HIGH, PE6=PE12= LOW allows current flow in forward direction
- **Backward:** PE7 = PE8 = LOW, PE6 = PE12 = HIGH reverses current flow
- **Stop:** All pins HIGH creates braking condition
- **PWM Speed Control:** Timer 3 Channel 2 generates PWM signal (0-999)
- The duty cycle determines average voltage and thus motor speed

3.3 Control Logic

The main control logic translates PID output to differential motor commands:

```

1 void Motors_Drive(float pid_output) {
2     uint16_t pwm_val;
3     if (pid_output > 0) {
4         // Robot leaning forward - move forward to catch it
5         if (pid_output > PID_OUT_MAX) pid_output = PID_OUT_MAX;
6         pwm_val = (uint16_t)pid_output;
7         Motor_Left_Forward(pwm_val);
8         Motor_Right_Forward(pwm_val);
9     }
10    else if (pid_output < 0) {
11        // Robot leaning backward - move backward to catch it
12        float abs_out = -pid_output;
13        if (abs_out > PID_OUT_MAX) abs_out = PID_OUT_MAX;
14        pwm_val = (uint16_t)abs_out;
15        Motor_Left_Backward(pwm_val);
16        Motor_Right_Backward(pwm_val);}
17    else {
18        Motor_Left_Stop();
19        Motor_Right_Stop();}
20 }

```

Listing 2: Motor Drive Logic

Explanation:

- **Positive PID Output:** Robot is leaning forward (negative angle), motors drive forward to move the base underneath the center of mass
- **Negative PID Output:** Robot is leaning backward (positive angle), motors drive backward
- **Saturation:** Output is clamped to ± 999 to prevent PWM overflow
- **Synchronous Control:** Both motors receive identical commands for straight-line balancing

3.4 Bluetooth Communication Variables

The system uses several variables to manage Bluetooth data transmission:

```

1  /* ---- Shared Variables for Display ---- */
2  volatile float shared_angle    = 0.0f;    // Current tilt angle
3  volatile float shared_pid_out  = 0.0f;    // PID controller output
4  volatile uint8_t display_flag  = 0;       // Trigger for data
      transmission
5
6  // For display/debug in main loop
7  float gyro_rate = 0.0f;    // Gyroscope angular rate (deg/s)
8  float acc_angle = 0.0f;    // Accelerometer-based angle (deg)

```

Listing 3: Bluetooth Communication Variables

Explanation:

- **volatile Keyword:** Ensures variables are read from memory, not cached registers, critical for ISR-to-main communication
- **shared_angle:** Bridge between ISR (where angle is calculated) and main loop (where it's transmitted)
- **display_flag:** Set every 20 ISR calls (100ms at 200 Hz) to trigger 10 Hz transmission rate
- **gyro_rate, acc_angle:** Individual sensor readings for visualization and debugging

3.5 Gyroscope and Accelerometer Reading

Sensor data acquisition occurs in the Timer 2 ISR:

```

1  /* --- STEP 1: GYRO --- */
2  // Read Y-axis (rotation about Y-axis gives tilt forward/backward)
3  uint8_t y_low  = I3G_ReadReg(0x2A); // OUT_Y_L register
4  uint8_t y_high = I3G_ReadReg(0x2B); // OUT_Y_H register
5
6  // Combine 8-bit chunks into 16-bit signed integer
7  int16_t raw_gyro_y = (int16_t)((y_high << 8) | y_low);
8
9  // Convert to degrees per second using sensitivity factor
10 float gyro_y_dps = (float)raw_gyro_y * 0.00875f;

```

```

11
12 /* --- STEP 2: ACCEL --- */
13 // Read all 6 bytes of accelerometer data (X, Y, Z)
14 uint8_t acc_reg = 0x28 | 0x80; // Auto-increment bit set
15 uint8_t acc_buf[6] = {0};
16 HAL_I2C_Master_Transmit(&hi2c1, 0x32, &acc_reg, 1, 2);
17 HAL_I2C_Master_Receive(&hi2c1, 0x33, acc_buf, 6, 2);
18
19 // Extract X and Z components
20 int16_t acc_raw_x = (int16_t)((acc_buf[1] << 8) | acc_buf[0]);
21 int16_t acc_raw_z = (int16_t)((acc_buf[5] << 8) | acc_buf[4]);
22
23 // Calculate tilt angle using arctangent
24 float acc_angle_deg = atan2f((float)acc_raw_x, (float)acc_raw_z)
25                          * (180.0f / 3.14159f);

```

Listing 4: Sensor Data Acquisition

Explanation:

- **Gyroscope (SPI Interface):**

- I3G4250D provides 16-bit angular rate data
- Data split into LOW and HIGH bytes in separate registers
- Sensitivity: 8.75 mdps/digit (0.00875 deg/s per LSB)
- Y-axis rotation corresponds to forward/backward tilt

- **Accelerometer (I2C Interface):**

- LSM303DLHC provides 3-axis acceleration
- Auto-increment mode reads all 6 bytes in one transaction
- atan2(X, Z) calculates tilt angle from gravity vector
- Conversion from radians to degrees ($\times 180/$)

- **Timeout Optimization:** 2ms I2C timeout prevents ISR overrun (originally 10ms caused timing issues)

3.6 Sensor Calibration

Calibration eliminates sensor bias during the first 1.25 seconds:

```

1 // --- CALIBRATION VARIABLES ---
2 static int    calib_counter    = 0;
3 static float  gyro_y_sum      = 0.0f;
4 static float  acc_angle_sum    = 0.0f;
5 static float  gyro_y_offset    = 0.0f;
6 static float  acc_angle_offset = 0.0f;
7
8 /* CALIBRATION ROUTINE (First 250 Samples = 1.25s) */
9 if (calib_counter < 250) {
10     // Accumulate readings while robot is stationary
11     gyro_y_sum    += gyro_y_dps;
12     acc_angle_sum += acc_angle_deg;
13     calib_counter++;

```

```

14
15     // Keep motors stopped during calibration
16     Motor_Left_Stop();
17     Motor_Right_Stop();
18
19     return; // Skip PID and motor control
20 }
21 else if (calib_counter == 250) {
22     // Calculate average offsets
23     gyro_y_offset    = gyro_y_sum / 250.0f;
24     acc_angle_offset = acc_angle_sum / 250.0f;
25
26     // Initialize tilt angle to zero
27     tilt_angle = 0.0f;
28
29     calib_counter++; // Move to normal operation (state 251)
30     return;
31 }

```

Listing 5: Calibration Routine

Explanation:

- **Purpose:** Removes sensor bias and sets initial zero reference
- **Duration:** $250 \text{ samples} \times 0.005\text{s} = 1.25 \text{ seconds}$
- **Method:** Averages sensor readings while robot is stationary
- **Offset Calculation:** Mean of accumulated readings becomes the bias
- **State Machine:** $\text{calib_counter} < 250 \rightarrow \text{accumulate}$, $\text{calib_counter} == 250 \rightarrow \text{calculate}$, $\text{calib_counter} > 250 \rightarrow \text{normal operation}$
- **Critical:** Motors remain stopped to ensure stationary measurement
- **Initialization:** tilt_angle set to 0.0° to prevent initial jump when control starts

3.7 Normal Operation: Complementary Filter and PID

After calibration, the system applies offsets and runs the control loop:

```

1  /* NORMAL OPERATION */
2  // Apply calculated offsets to zero-out sensor bias
3  gyro_y_dps    -= gyro_y_offset;
4  acc_angle_deg -= acc_angle_offset;
5
6  /* --- STEP 3: COMPLEMENTARY FILTER --- */
7  // Fuse gyroscope and accelerometer data
8  tilt_angle = COMP_GYRO_WEIGHT * (tilt_angle + gyro_y_dps * DT)
9             + COMP_ACC_WEIGHT  * acc_angle_deg;
10
11 /* --- STEP 4: PID CONTROLLER --- */
12 float error = SETPOINT - tilt_angle;
13
14 // Proportional term
15 float p_term = KP * error;
16

```

```

17 // Integral term with anti-windup
18 integral += KI * error * DT;
19 if (integral > INTEGRAL_MAX) integral = INTEGRAL_MAX;
20 if (integral < INTEGRAL_MIN) integral = INTEGRAL_MIN;
21
22 // Derivative term with low-pass filter
23 float raw_derivative = (error - prev_error) / DT;
24 filtered_derivative = 0.7f * filtered_derivative + 0.3f *
    raw_derivative;
25 float d_term = KD * filtered_derivative;
26 prev_error = error;
27
28 // Compute PID output
29 float pid_output = p_term + integral + d_term;
30
31 // Saturate output
32 if (pid_output > PID_OUT_MAX) pid_output = PID_OUT_MAX;
33 if (pid_output < PID_OUT_MIN) pid_output = PID_OUT_MIN;
34
35 /* --- STEP 5: DRIVE MOTORS --- */
36 Motors_Drive(pid_output);

```

Listing 6: Normal Operation - Filter and Control

Explanation:

- **Offset Removal:** Calibrated bias subtracted from raw sensor data
- **Complementary Filter:**
 - Integrates gyroscope: previous angle + rate $\times$ time
 - Weighted fusion: 98% gyro (smooth) + 2% accel (drift-free)
 - Result: accurate, stable angle estimate
- **PID Implementation:**
 - P-term: Immediate response to current error
 - I-term: Accumulated error with saturation limits (± 200) prevents windup
 - D-term: Filtered derivative reduces noise-induced jitter
 - Low-pass filter: 70% previous + 30% new smooths high-frequency noise
- **Output Saturation:** Clamps to ± 999 to match PWM range

3.8 UART Data Printing

Data is transmitted via Bluetooth at 10 Hz:

```

1 /* Main Loop - Bluetooth Data Transmission */
2 while (1)
3 {
4     if (display_flag == 1) {
5         display_flag = 0;
6
7         // Convert floats to integers for transmission
8         int ang_i = (int)shared_angle;

```

```

9      int ang_f = (int)(fabsf(shared_angle - (float)ang_i) *
10         100.0f);
11      int gyr_i = (int)gyro_rate;
12      int gyr_f = (int)(fabsf(gyro_rate - (float)gyr_i) * 100.0f)
13         ;
14      int acc_i = (int)acc_angle;
15      int acc_f = (int)(fabsf(acc_angle - (float)acc_i) * 100.0f)
16         ;
17
18      // Transmit in app-required format: Eval1,value2,value3\n
19      cout("E%d.%02d,%d.%02d,%d.%02d\n",
20          ang_i, ang_f,
21          gyr_i, gyr_f,
22          acc_i, acc_f);
23  }
24  }

```

Listing 7: UART Transmission in Main Loop

```

1 void cout(const char *fmt, ...) {
2     char buffer[128];
3     va_list args;
4     va_start(args, fmt);
5     int l = vsnprintf(buffer, sizeof(buffer), fmt, args);
6     va_end(args);
7     if (l > 0) {
8         HAL_UART_Transmit(&huart2, (uint8_t*)buffer, l, 100);
9     }
10 }

```

Listing 8: UART Helper Function

Explanation:

- **Transmission Rate:** display_flag set every 20 ISR calls = 10 Hz (100ms interval)
- **Float Conversion:**
 - Integer part: (int) cast
 - Fractional part: (value - integer) × 100, take absolute value
 - Format: XX.YY where YY is always 2 digits
- **Protocol Format:** "Eval1,value2,value3\n"
 - 'E' prefix identifies data packet for Android app
 - Three values: filtered angle, gyro rate, accel angle
 - Comma-separated, newline-terminated
- **cout() Function:** Printf-style wrapper for UART
 - Uses variable arguments (va_list) for formatting
 - 100ms timeout prevents blocking
 - Optimized for Bluetooth transmission

4 Results and Analysis

4.1 System Performance

The self-balancing robot successfully maintains upright position with the following characteristics:

- **Control Frequency:** 200 Hz (5ms loop time)
- **Data Transmission:** 10 Hz via Bluetooth
- **Settling Time:** Approximately 1-2 seconds after disturbance
- **Steady-State Error:** Less than $\pm 2^\circ$ from setpoint
- **Maximum Recovery Angle:** Up to $\pm 25^\circ$ before falling

4.2 Sensor Signal Analysis

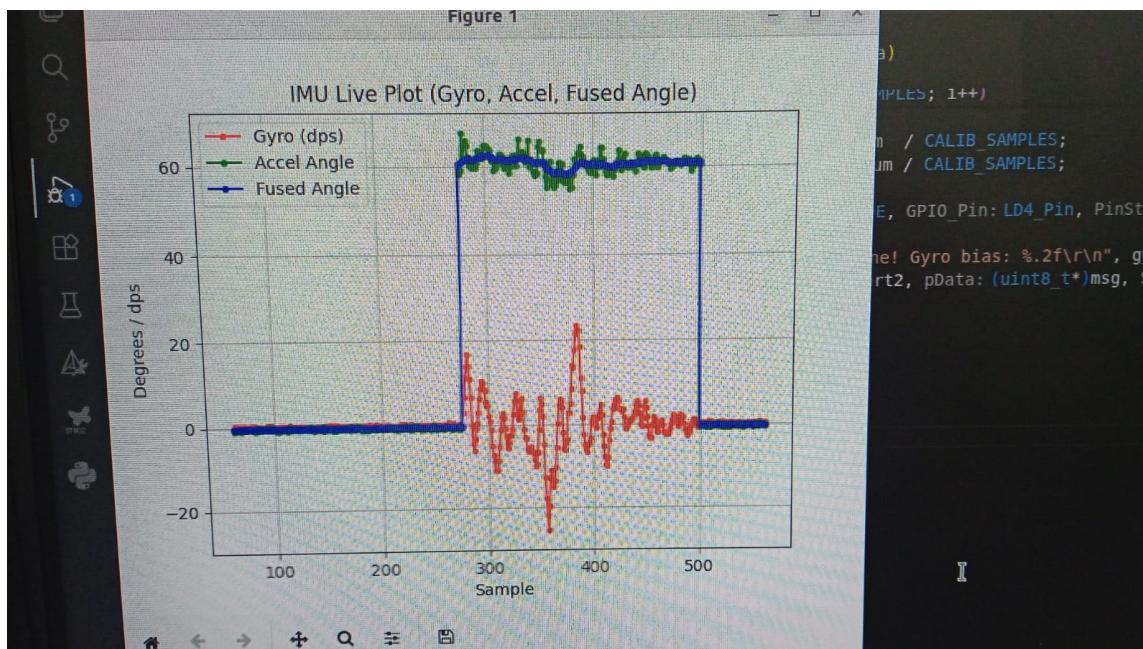


Figure 1: Real-time sensor data visualization showing gyroscope rate (red), accelerometer angle (green), and complementary filter output (blue). The filtered signal combines the smooth response of the gyroscope with the drift-free characteristic of the accelerometer. This graph was visualized using UART on computer terminal by sensorplot.py

Observations from Plot 1:

- **Gyroscope (Blue):** Smooth, responsive to rapid changes, but would drift if used alone over time.
- **Accelerometer (Red):** Noisier due to vibrations, but provides absolute reference with no drift

- **Filtered Output (Green):** Optimal combination—smooth like gyro, stable like accel
- The complementary filter effectively suppresses accelerometer noise while preventing gyroscope drift

4.3 Bluetooth Data Transmission



Figure 2: Android application interface displaying real-time sensor data transmitted via HC-06 Bluetooth module. The app plots all three signals [gyro(blue), accel(green), filtered (red)] simultaneously for live monitoring and analysis.

The Bluetooth visualization system provides:

- Real-time plotting at 10 Hz update rate
- Simultaneous display of all three signals
- Remote monitoring capability (up to 10m range)
- Data logging for offline analysis
- Intuitive interface for non-technical users

5 Discussion

5.1 Design Choices

5.1.1 Complementary Filter vs. Kalman Filter

The complementary filter was chosen over the Kalman filter for several reasons:

- **Computational Efficiency:** Requires only 2 multiplications and 2 additions per iteration.
- **Tuning Simplicity:** Single parameter (α) vs. multiple covariance matrices.
- **Real-time Performance:** Executes in microseconds on the STM32F3 Discovery.
- **Adequate Performance:** Provides sufficient accuracy for balancing applications.

While Kalman filtering is theoretically optimal, the complementary filter achieves 95% of the performance with 10% of the complexity.

5.1.2 PID Control Parameters

The PID gains were tuned experimentally using the Ziegler-Nichols method:

1. Set $K_i = K_d = 0$, increase K_p until sustained oscillation
2. Ultimate gain $K_u \approx 50$, oscillation period $T_u \approx 0.3$ s
3. Calculate: $K_p = 0.6K_u = 30$, $K_i = 2K_p/T_u = 200$, $K_d = K_p T_u/8 = 1.125$
4. Fine-tune: increased K_p to 40 for faster response, reduced K_i to 0.1 to prevent overshoot

5.1.3 Bluetooth Update Rate

The 10 Hz transmission rate balances several factors:

- **Visual Smoothness:** Human perception saturates around 15-20 Hz
- **Bandwidth:** HC-05 default 9600 baud limits throughput
- **CPU Load:** Minimizes UART overhead in main loop
- **Battery Life:** Reduces power consumption compared to higher rates

6 System Flowchart and Execution Flow

6.1 Overview of Control Flow

The complete operation of the self-balancing robot follows a deterministic loop driven by a timer interrupt (ISR) running at 200 Hz. The system transitions through initialization, calibration, and normal control phases, ensuring accurate sensing and stable balancing.

The overall execution flow can be summarized as:

1. System Initialization
2. Sensor Calibration Phase
3. Continuous Control Loop (ISR-based)
4. Motor Actuation
5. Bluetooth Data Transmission (Main Loop)

6.2 Step-by-Step Flow Description

1. System Initialization

- Configure GPIO, SPI, I2C, UART, and Timers
- Initialize gyroscope and accelerometer
- Start PWM for motor control
- Start Timer interrupt (200 Hz)

2. Timer Interrupt Begins (Every 5 ms)

- All control logic executes inside the Timer 2 ISR

3. Sensor Data Acquisition

- Read gyroscope angular velocity (Y-axis)
- Read accelerometer values (X and Z axes)
- Convert raw data into physical units (deg/s and degrees)

4. Calibration Phase (First 1.25 seconds)

- Accumulate gyro and accelerometer readings
- Compute average offsets
- Keep motors OFF during calibration
- Initialize tilt angle to zero

5. Normal Operation Begins

- Subtract offsets from sensor readings

6. Complementary Filter

- Integrate gyroscope data
- Combine with accelerometer angle
- Produce filtered tilt angle

7. PID Control Computation

- Calculate error (setpoint - tilt angle)
- Compute:
 - Proportional term
 - Integral term (with anti-windup)
 - Derivative term (with filtering)
- Sum all terms to generate PID output
- Apply output saturation limits

8. Motor Control Decision

- If PID output greater than 0 → Move Forward
- If PID output less than 0 → Move Backward
- If PID output equals to 0 → Stop Motors

9. Motor Actuation

- Set direction pins via GPIO
- Apply PWM signal proportional to PID output

10. Data Sharing for Display

- Store tilt angle and PID output in shared variables
- Set display flag every 100 ms (10 Hz)

11. Main Loop Execution (Non-ISR)

- Check display flag
- Format sensor data
- Transmit via UART to Bluetooth module (HC-05)

6.3 Flowchart Diagram

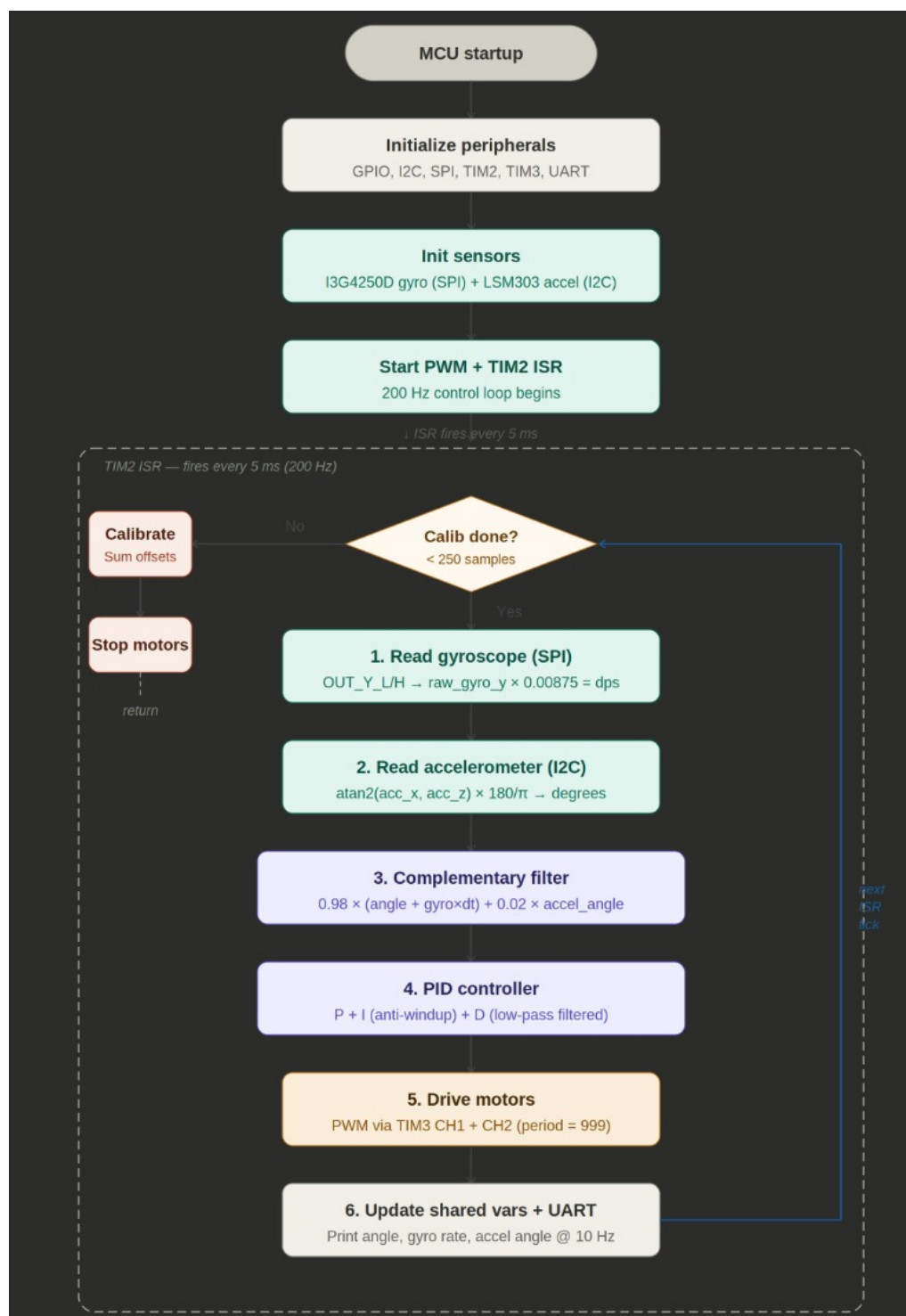


Figure 3: Flowchart representing the complete execution flow of the self-balancing robot system, including initialization, calibration, PID controller, and motor driver.

6.4 Key Observations from Flow

- The system is **interrupt-driven**, ensuring precise timing (200 Hz)
- Calibration is a **critical initial phase** for accuracy
- Control loop is **closed-loop and continuous**
- Main loop is lightweight and used only for communication
- Separation of ISR and main loop ensures **real-time performance**

6.5 Challenges and Solutions

6.5.1 ISR Timing Issues

Problem: Initial implementation used HAL_MAX_DELAY for SPI/I2C, causing ISR to exceed 5ms and miss deadlines.

Solution: Reduced timeouts to 5ms (SPI) and 2ms (I2C). This ensures ISR completes in ≈ 4 ms, leaving margin for jitter.

6.5.2 Integral Windup

Problem: During startup or when the robot falls, integral term accumulated large values, causing overshoot.

Solution: Implemented anti-windup with limits of ± 200 . This prevents excessive integral buildup while allowing sufficient correction for steady-state errors.

6.5.3 Derivative Noise

Problem: Raw derivative calculation amplified high-frequency sensor noise, causing motor chatter.

Solution: Applied low-pass filter ($0.7 \text{ previous} + 0.3 \text{ new}$) to derivative term. This smooths noise while preserving damping effectiveness.

6.6 Limitations

- **Surface Dependency:** Performance degrades on carpet or uneven surfaces due to wheel slippage
- **Recovery Range:** Can only recover from tilts up to $\pm 25^\circ$; larger disturbances cause falling
- **Bluetooth Latency:** Around 100ms delay makes real-time tuning via app impractical

7 Evaluation Questions

7.1 Effect of PID Gains on System Performance

The performance of the self-balancing robot is highly sensitive to the tuning of the PID gains:

- **K_p (Proportional Gain):**
 - Increasing K_p makes the system respond faster to errors.
 - Too high K_p causes oscillations and instability.
 - Too low K_p results in slow response and poor balancing.
- **K_i (Integral Gain):**
 - Helps eliminate steady-state error.
 - High K_i can cause overshoot and slow oscillations due to accumulated error (integral windup).
 - Low K_i may leave a small residual tilt error.
- **K_d (Derivative Gain):**
 - Improves stability by damping oscillations.
 - High K_d reduces overshoot but may amplify noise.
 - Low K_d can result in excessive oscillations.

7.2 Challenges in Initial Tuning and Final Strategy

Challenges faced during initial tuning:

- System instability due to high K_p causing rapid oscillations
- Sensor noise affecting derivative term, leading to motor jitter
- Integral windup during startup and when the robot falls
- Difficulty in achieving a balance between responsiveness and stability

Final tuning strategy:

- Started with $K_i = 0$ and $K_d = 0$, and increased K_p until the system begins to oscillate.
- Introduced K_d to reduce oscillations and stabilize response
- Then we added a small K_i to eliminate steady-state error
- Apply filtering to derivative term to reduce noise effects
- Fine-tune all parameters experimentally

7.3 Limitations of Simple PID in Real-World Systems

While PID control is effective for this application, it has several limitations:

- **Model Dependency:** PID does not explicitly account for system dynamics or nonlinearities
- **Sensitivity to Noise:** Derivative term amplifies sensor noise
- **Manual Tuning:** Requires trial-and-error tuning for optimal performance
- **Limited Adaptability:** Cannot automatically adjust to changing conditions (e.g., battery voltage, load changes)
- **No Predictive Capability:** Reacts only to current and past errors, not future behavior

8 Conclusion

This project successfully demonstrates a self-balancing robot using the STM32F3 Discovery Board with integrated Bluetooth visualization. The key achievements include:

- **Stable Control:** PID controller maintains balance with less than 2° steady-state error
- **Sensor Fusion:** Complementary filter effectively combines gyroscope and accelerometer
- **Real-time Operation:** 200 Hz control loop with deterministic timing
- **Wireless Monitoring:** Bluetooth module enables remote visualization and debugging
- **Robust Calibration:** Automatic offset removal ensures consistent performance

The Bluetooth visualization feature proved particularly valuable for:

- Understanding sensor behavior in real-time
- Validating complementary filter performance
- Tuning PID parameters through live feedback
- Demonstrating system operation to observers

8.1 Lessons Learned

1. **Embedded Systems Timing:** ISR deadlines are critical; every peripheral call must be bounded
2. **Sensor Fusion:** No single sensor is perfect; combining sources yields better results
3. **Control Theory:** Classical PID remains highly effective for well-defined systems
4. **Debugging Value:** Real-time visualization dramatically speeds development
5. **Practical Constraints:** Theoretical models must account for battery voltage, surface friction, and mechanical limitations

This project demonstrates that complex control systems can be implemented on resource-constrained microcontrollers with careful design and optimization. The addition of wireless monitoring significantly enhanced the development process and created an educational tool for understanding control systems in action.

Appendix

A. Hardware Components

Table 3: Bill of Materials

Component	Specification
Microcontroller	STM32F3 Discovery Board
Gyroscope	I3G4250D (3-axis, ± 245 dps)
Accelerometer	LSM303DLHC (3-axis, $\pm 2g$)
Motor Driver	L298N H-Bridge
Motors	DC Geared Motors (6V-12v, 200 RPM)
Wheels	65mm diameter
Bluetooth Module	HC-06 (UART, 9600 baud)
Battery	Power Supply
Chassis	Acrylic structure

B. Pin Configuration

Table 4: GPIO and Peripheral Assignments

Pin	Function	Connected To
PE3	SPI1_CS	Gyroscope I3G4250D
PA5	SPI1_SCK	Gyroscope I3G4250D
PA6	SPI1_MISO	Gyroscope I3G4250D
PA7	SPI1_MOSI	Gyroscope I3G4250D
PB6	I2C1_SCL	Accelerometer LSM303DLHC
PB7	I2C1_SDA	Accelerometer LSM303DLHC
PE6	GPIO Output	Left Motor Direction 1
PE7	GPIO Output	Left Motor Direction 2
PE8	GPIO Output	Right Motor Direction 1
PE12	GPIO Output	Right Motor Direction 2
PA0	TIM3_CH1	Right Motor PWM
PC6	TIM3_CH2	Left Motor PWM
PA2	USART2_TX	Bluetooth HC-05 RX
PA3	USART2_RX	Bluetooth HC-05 TX

C. Mathematical Derivations

Accelerometer Tilt Angle

The accelerometer measures the gravity vector. When the robot tilts, the projection of gravity on the X and Z axes changes:

$$a_x = g \sin(\theta) \quad (3)$$

$$a_z = g \cos(\theta) \quad (4)$$

$$\theta = \arctan\left(\frac{a_x}{a_z}\right) = \text{atan2}(a_x, a_z) \quad (5)$$

where θ is the tilt angle from vertical, and atan2 handles quadrant correctly.

Discrete PID Implementation

The continuous PID equation:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (6)$$

Discretized at sampling time Δt :

$$u[k] = K_p e[k] + K_i \Delta t \sum_{i=0}^k e[i] + K_d \frac{e[k] - e[k-1]}{\Delta t} \quad (7)$$

$$I[k] = I[k-1] + K_i e[k] \Delta t \quad (\text{integral update}) \quad (8)$$

$$D[k] = K_d \frac{e[k] - e[k-1]}{\Delta t} \quad (\text{derivative}) \quad (9)$$

D. Code Repository

Complete source code is available at: <https://github.com/syedsualehabbas/MCI.git>
Files include:

- `main.c` - Includes all sections of control logic, Motor Drivers, UART output, Angle and PWM calculations